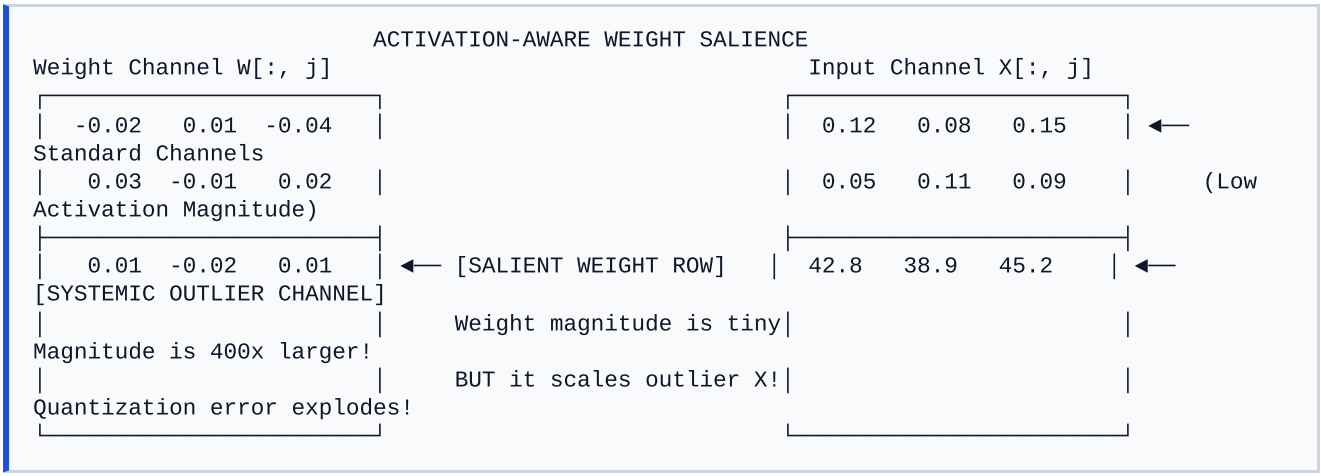
TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 11 (Part 2): Quantization & Inference Optimization

Module 11 Part 2 examines low-bit quantization algorithms, file containers, runtime frameworks, and hardware acceleration techniques: from Activation-aware Weight Quantization (AWQ) and GGUF block layouts to bitsandbytes memory mechanics, speculative decoding, and hardware-specific deployment profiling.

11.9 AWQ (Activation-Aware Weight Quantization): Theory & Formulation

Introduced by Lin et al. (MIT, MLSys 2024), **AWQ (Activation-aware Weight Quantization)** addresses a core weakness of traditional post-training quantization: weight parameters should not be evaluated in isolation. A weight's importance to downstream task accuracy depends on the **magnitude of the activations flowing through it**.



1. Why Activation Magnitude Matters More Than Weight Magnitude

Earlier pruning and quantization algorithms assumed that weights with larger absolute magnitudes ($|w|$) were more important. AWQ demonstrated that:

1. **Weight Magnitude is Deceptive:** Many weights with small magnitudes ($|w| \approx 0.01$) interact directly with outlier activation channels ($|x| \approx 50.0$).
2. **Amplification of Truncation Noise:** Consider linear matrix multiplication:

   $$Y = X \cdot W = \sum_{j=1}^{d_{\text{in}}} X_{:,j} \cdot W_{j,:}$$

   If a quantization error $\Delta W = W - \hat{W}$ occurs at input channel j, the resulting error in output activations is:

   $$\Delta Y = X_{:,j} \cdot \Delta W_{j,:}$$

   If channel j carries an outlier activation ($X_{:,j} \gg 1$), even a tiny rounding error in $W_{j,:}$ is , degrading downstream layer outputs and compounding across deep Transformer blocks.

3. **amplified by the magnitude of $X_{:,j}$**

2. Identifying Salient Weights via Average Activation Magnitude

Rather than relying on second-order Hessian inversions ($H = 2XX^T$, which can become ill-conditioned around outliers), AWQ evaluates the channel-wise average activation magnitude:

$$s_X(j) = \frac{1}{N} \sum_{i=1}^{N} |X_{i,j}|$$

Weights corresponding to the top *1%* largest values of s_X are designated as **salient weights**. Protecting just this *1%* of weights prevents output degradation.

3. Per-Channel Equivalent Transformation (Protection via Activation Scaling)

Keeping 1% of weights in uncompressed FP16 creates mixed-precision tensor layouts that trigger memory fragmentation and break tensor-core GEMM throughput. AWQ avoids mixed precision through an **equivalent algebraic transformation**:

```
                        PER-CHANNEL EQUIVALENT SCALING
Original Matrix Multiply:
       Y = X · W = X · (S^-1 · S) · W = (X · S^-1) · (S · W) = X_scaled · W_scaled

Where S = diag(s_1, s_2, ..., s_d) is a positive per-channel diagonal scaling matrix.
```

By multiplying the weight matrix by a diagonal scale matrix S while dividing incoming activations by the exact same scale S:

$$Y = XW = (XS^{-1})(SW) = \hat{X}\hat{W}$$

- **Mathematical Invariance:** The mathematical result of the linear layer is unchanged.
- **Quantization Smoothing:** By selecting an optimal scale vector $\boldsymbol{s} \in \mathbb{R}^{d_{in}}$, salient weights are scaled up:

  $$W'_{j,:} = W_{j,:} \cdot s_j \quad (s_j > 1)$$

  Because $W'_{j,:}$ is larger, its relative quantization step size shrinks:

  $$\text{Relative Rounding Error} = \frac{\text{Round}(W'_{j,:} / \text{Scale}) - (W'_{j,:} / \text{Scale})}{W'_{j,:}} \longrightarrow 0$$

  Scaling shifts the dynamic range so that rounding noise on salient channels is suppressed.

- **Finding the Optimal Scale:** AWQ finds the per-channel scale parameter s_j via grid search over a bounded parameter grid:

  $$s = s_X^{\gamma} \quad \text{where} \quad \gamma \in [0, 1]$$

  $$\arg\min_{\gamma} \| W X - \text{dequant}(\text{quant}(W \cdot \text{diag}(s_X^{\gamma}))) \cdot \text{diag}(s_X^{-\gamma}) X \|_2^2$$

  The search tests points in the range $\gamma \in [0, 1]$ (where $\gamma = 0$ is unscaled RTN, and $\gamma = 1$ is pure activation-proportional scaling). The optimal γ typically settles near $\boldsymbol{\gamma \approx 0.5}$ (geometric mean scaling).

- **Zero Runtime Overhead:** At inference time, the division $X S^{-1}$ is fused directly into the weight matrix of the **preceding layer** (e.g., the LayerNorm or RMSNorm scaling vectors), adding **zero additional FLOPs or runtime latency**.

11.10 AWQ vs. GPTQ: Architectural, Speed, and Quality Comparison

Both AWQ and GPTQ achieve 4-bit weight compression on LLMs, but their optimization mechanisms and hardware behavior diverge.

```
┌──────────────────────────────────────┬──────────────────────────────────────┐
│     GPTQ (SECOND-ORDER OBS)          │     AWQ (EQUIVALENT SCALING)         │
├──────────────────────────────────────┼──────────────────────────────────────┤
│ • Inverts Hessian (H^-1 = 2XX^T)     │ • Measures activation energy         │
│ • Mutates ALL unquantized            │ • Preserves original weight          │
│   weights to compensate for          │   distribution; scales salient       │
│   rounding error (OBS update)        │   channels by S = s_X^γ              │
│ • Higher risk of overfitting to      │ • Highly generalizable; zero         │
│   calibration data distributions     │   overfitting to calibration         │
│ • Complex Act-Order shuffles         │ • Native contiguous memory;          │
│   memory layouts                     │   optimal for fast GPU GEMM          │
└──────────────────────────────────────┴──────────────────────────────────────┘
```

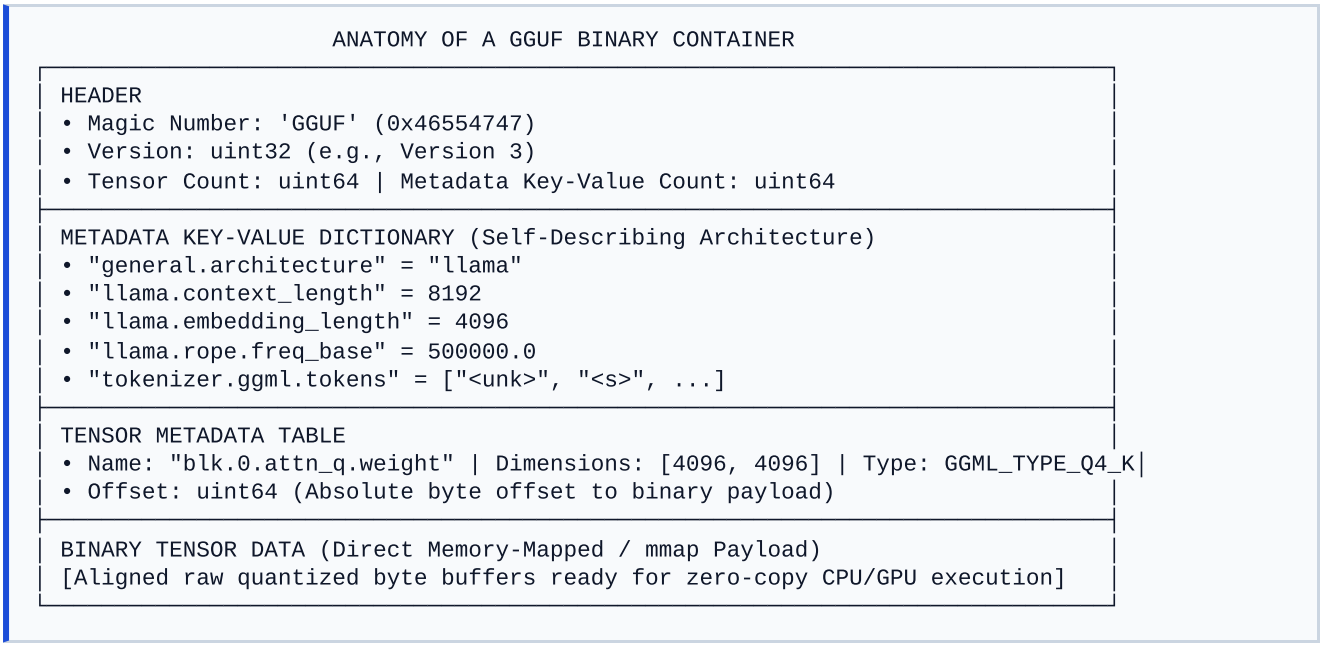
Head-to-Head Comparison Matrix

Evaluation Dimension	GPTQ (Frantar et al.)	AWQ (Lin et al.)
Underlying Mathematics	Second-order Taylor series; inverse Hessian Cholesky updates.	First-order activation observation; per-channel equivalent grid search.
Weight Modification	**Aggressively mutates weights:** Changes unquantized values to absorb error.	**Zero structural distortion:** Multiplies channels by scale factor S; preserves weight manifold.
Calibration Sensitivity	**High:** Overfits if calibration text diverges from target domain.	**Low:** Activation channel distributions remain consistent across domains.
Act-Order Requirement	Often requires `act-order=True` to preserve reasoning on 7B/8B models.	**None:** Preserves natural channel ordering natively.
Inference Kernel Alignment	Non-contiguous memory access (if Act-Order is on) can degrade GPU memory coalescing.	**Hardware-Friendly:** Grouped weights remain contiguous, mapping cleanly to SIMD registers.

Evaluation Dimension	GPTQ (Frantar et al.)	AWQ (Lin et al.)
Quantization Speed	Moderate (~1.5 hours for 70B on 1x A100).	**Fast** (~20–30 minutes for 70B on 1x A100).
Engine Support	AutoGPTQ, vLLM, TensorRT-LLM, ExLlamaV2.	vLLM, AutoAWQ, TensorRT-LLM, SGLang, TGI.
Production Recommendation	Recommended when deploying via **ExLlamaV2** on consumer hardware.	**Recommended production default** for server-side serving (vLLM / SGLang).

11.11 GGUF Format and llama.cpp Quantization Schemes Explained

Developed by Georgi Gerganov and the `llama.cpp` team, **GGUF (GPT-Generated Unified Format)** is the standard binary container for local, edge, and CPU/GPU-offloaded LLM execution. It replaced GGML to address layout inflexibility and metadata brittleness.



1. The GGUF Architecture

- **Single-File Encapsulation:** Contains everything needed to run the model: hyperparameters, full tokenizer vocabularies, chat templates, and quantized weights.
- **Memory-Mapped I/O (`mmap`):** Tensors are aligned to 32-byte boundaries. Runtimes mount the model from NVMe drives directly into system virtual memory without parsing overhead, enabling fast startup times and dynamic memory sharing across OS processes.
- **Extensibility:** The key-value metadata schema allows new architectural hyperparameters (such as custom RoPE parameters, MoE routing coefficients, and vision encoders) to be embedded without breaking older parsers.

2. llama.cpp Quantization Levels: Legacy vs. K-Quants

`llama.cpp` uses block-based quantization schemes, processing weights in contiguous blocks of 32 or 256 values.

```
      K-QUANTS BLOCK ARCHITECTURE (SUPER-BLOCK OF 256 VALUES)
┌──────────────────────────────────────────────────────────────────────────┐
│ SUPER-BLOCK (256 Weights)                                                │
│ ├─ Sub-block 0 (32 values): Quantized to low bits (e.g., 4 bits per weight)  │
│ ├─ Sub-block 1 (32 values): Quantized to low bits                        │
│ ├─ ...                                                                   │
│ ├─ Sub-block 7 (32 values): Quantized to low bits                        │
│ └─ Scale / Min metadata: Quantized to 6 bits or 8 bits (Double Quantization) │
└──────────────────────────────────────────────────────────────────────────┘
```

The Legacy Quantization Schemes (`Q4_0` , `Q4_1` , `Q8_0`)

- **`Q4_0`:** Symmetric 4-bit quantization. Block size = 32. Each block stores 32 4-bit values (16 bytes) plus one FP16 scale factor (2 bytes).

  $$\text{Effective bpb} = 4 + \frac{16}{32} = 4.5 \text{ bits per weight}$$

- **`Q4_1`:** Asymmetric 4-bit quantization. Block size = 32. Stores 32 4-bit values, an FP16 scale, and an FP16 minimum value.

  $$\text{Effective bpb} = 4 + \frac{32}{32} = 5.0 \text{ bits per weight}$$

- **`Q8_0`:** Symmetric 8-bit quantization. Block size = 32. Yields an effective 8.5 bpb. Provides near-perfect floating-point parity and is commonly used to quantize embedding matrices and the final `output.weight` layer.

Modern K-Quants Hierarchy (Super-Blocks of 256 Weights)

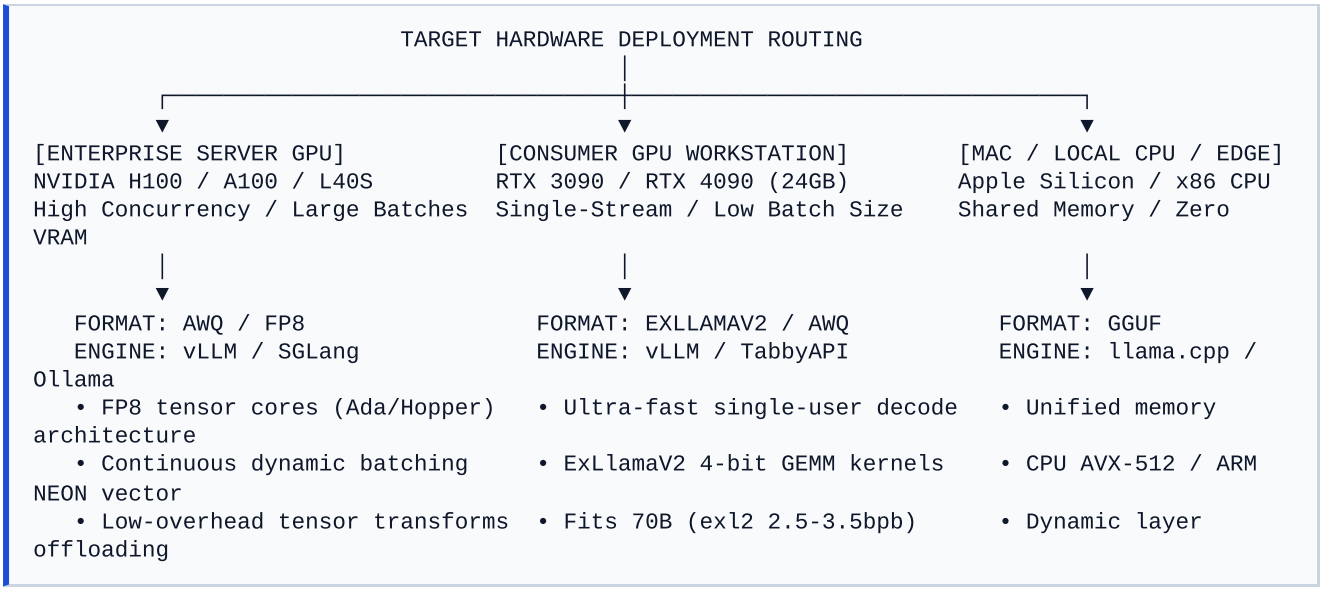
K-Quants introduce **multi-level block hierarchies** to shrink scale and offset metadata overhead:

- **`Q2_K`:** 2-bit quantization using 16-weight sub-blocks with 4-bit scales. Highly degraded perplexity; reserved for small edge devices where larger models cannot otherwise fit into memory.
- **`Q3_K_S`** / **`Q3_K_M`** / **`Q3_K_L`:** 3-bit variations. `Q3_K_M` uses 3-bit weights for attention layers and low-impact MLPs, retaining 4-bit precision for critical feed-forward projection matrices.
- **`Q4_K_M` (The Standard Edge Baseline):**
  - Divides 256 weights into 8 sub-blocks of 32.
  - Weights are quantized to 4 bits.
  - The scale factors and offsets for the 8 sub-blocks are themselves quantized to 6 bits (hierarchical compression).
  - Critical attention matrices (`v_proj` , `o_proj` , and part of `feed_forward`) are allocated 5-bit or 6-bit weights.
  - Yields ~4.5 bits per weight while preserving ~99% of FP16 benchmark performance on 7B–70B models.

- `Q5_K_M` **(The High-Fidelity Standard):**
  - Uses 5-bit weights across key layers, paired with 6-bit scales.
  - Yields ~5.5 bits per weight.
  - Fixes subtle degradation in multi-step mathematical deduction and deep code generation where 4-bit quantization falls short.
- `Q6_K` **:** 6-bit quantization throughout. Perplexity matches uncompressed 16-bit float baselines within statistical noise margins.

11.12 Choosing a Quantization Format for a Target Deployment

Deploying quantized models requires matching the quantization scheme and container format to the physical compute and memory characteristics of the target hardware.



Comprehensive Deployment Decision Framework

Target Hardware Environment	Primary Latency / Compute Bottleneck	Optimal Quantization Format	Optimal Serving Runtime Engine	Key Configuration Directives
Data-Center Multi-GPU (A100 / H100)	High concurrent query volume; network cross-node bandwidth.	**FP8 (Native E4M3 / E5M2) or AWQ (INT4)**	**vLLM / TensorRT-LLM**	Continuous PagedAttention batching; tensor parallelism (`tp=2/4/8`).
Enterprise Single-GPU (NVIDIA L4 / A10G / L40S)	VRAM ceiling (24GB); compute-bound prefill phase.	**AWQ (INT4, group=128)**	**vLLM / SGLang**	Use AWQ with fused flash-attention kernels to maximize multi-tenant concurrency.
Consumer Desktop GPU (RTX 4090 / 3090 - 24GB)	Memory capacity limits (cannot fit 70B FP16); memory bus throughput.	**EXL2 (2.2 – 4.0 bpb) or AWQ**	**ExLlamaV2 / vLLM**	Quantize to ~3.5 bpb to fit 70B models into 24GB VRAM with an 8k context window.

Target Hardware Environment	Primary Latency / Compute Bottleneck	Optimal Quantization Format	Optimal Serving Runtime Engine	Key Configuration Directives
Apple Silicon (M2/M3/M4 Max/Ultra - 64GB–192GB)	Memory bandwidth limits; lack of dedicated CUDA cores.	**GGUF (`Q4_K_M` or `Q5_K_M`)**	**llama.cpp / Ollama (Metal Engine)**	Load model into unified RAM; Metal framework maps matrix-vector products directly to GPU cores.
x86 Enterprise CPU Servers (Xeon / EPYC)	Compute latency; lacks discrete parallel GPU accelerators.	**GGUF (`Q4_K_M` or `Q8_0`)**	**llama.cpp**	Compile with native `AVX-512` and `AMX` instructions; match thread count to physical CPU cores.

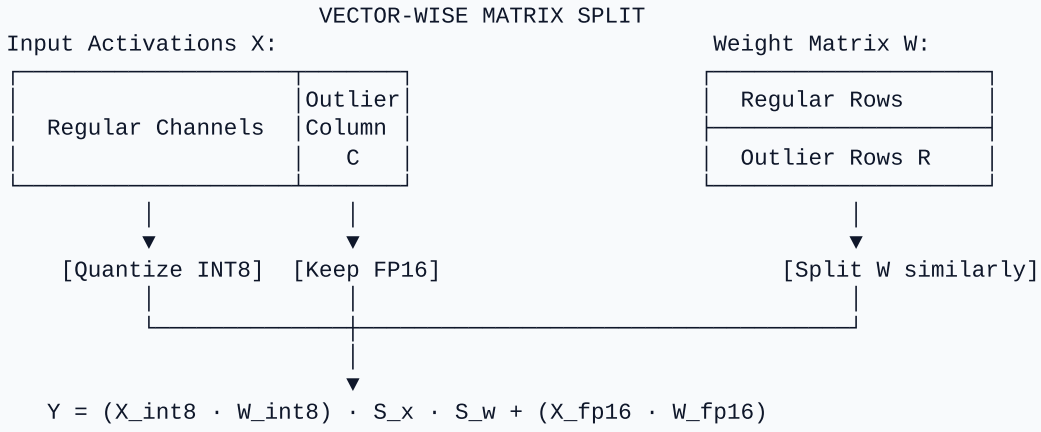
11.13 The `bitsandbytes` Library: 8-Bit and 4-Bit (NF4) Mechanics

Developed by Tim Dettmers, **`bitsandbytes`** is the standard library for drop-in, zero-configuration mixed-precision inference and fine-tuning across the Hugging Face ecosystem.

```
┌──────────────────────────────────────┬──────────────────────────────────────┐
│     LLM.int8() (VECTOR-WISE)         │      4-BIT NF4 INFERENCE             │
├──────────────────────────────────────┼──────────────────────────────────────┤
│ • Outlier-Aware Vector Scaling       │ • Information-theoretically          │
│ • Splits activations:                │   optimal NormalFloat4 mapping       │
│   Regular channels ──► INT8          │ • Stored statically in 4 bits        │
│   Outlier channels ──► FP16          │ • Dequantized dynamically into       │
│ • Fuses both matrix products         │   BF16/FP16 within SRAM blocks       │
│ • Preserves 100% of perplexity       │ • Ultra-low VRAM memory ceiling      │
└──────────────────────────────────────┴──────────────────────────────────────┘
```

1. `LLM.int8()` : Vector-Wise Outlier Decomposition

Standard INT8 quantization breaks down when applied to LLMs larger than 6.7B parameters due to activation outliers. `bitsandbytes` resolves this via **Vector-Wise Outlier Decomposition**:



1. Given input activations X, the runtime identifies outlier channels containing values $|x| \geq$ threshold (typically threshold = 6.0).
2. The columns containing outliers are extracted into an unquantized FP16 matrix $X_{fp16} \in \mathbb{R}^{T \times k}$ (where $k \ll d_{in}$). The remaining *99.9%* of channels are quantized into an 8-bit integer matrix X_{int8}.
3. The weight matrix W is similarly split into regular 8-bit components W_{int8} and unquantized 16-bit rows W_{fp16}.
4. **Dual Computation:**
   - The bulk of operations run through high-speed INT8 tensor cores: $Y_{int8} = (X_{int8} \cdot W_{int8}) \cdot S_X S_W$.
   - Outlier channels run through standard FP16 cores: $Y_{fp16} = X_{fp16} \cdot W_{fp16}$.
5. The outputs are summed: $Y = Y_{int8} + Y_{fp16}$.
6. **Result:** Completely avoids quantization-induced degradation across models up to 175B+ parameters.

2. 4-Bit Inference (NF4 Engine)

During inference with `load_in_4bit=True`, `bitsandbytes` loads base weights in **4-bit NormalFloat (NF4)**:

- Weights are packed at 2 elements per byte in global VRAM.
- For each linear forward pass, blocks of weights are loaded into GPU on-chip SRAM, dequantized on the fly into 16-bit floats (BF16/FP16), multiplied by incoming activations, and immediately discarded from SRAM.
- **Production Implication:** Slashes base model memory by **~70%**. However, because weights must be dequantized from 4-bit to 16-bit before computation (rather than executing directly in 4-bit integer tensor cores), **`bitsandbytes` 4-bit inference is slower than native 4-bit runtimes like AWQ or ExLlamaV2**, which use specialized, pre-compiled fused INT4 GEMM kernels.

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# 1. Configure bitsandbytes 4-bit quantization profile
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",               # Optimal normal distribution mapping
    bnb_4bit_use_double_quant=True,          # Quantize scales to 0.127 bpb
    bnb_4bit_compute_dtype=torch.bfloat16    # Target precision for linear compute
)

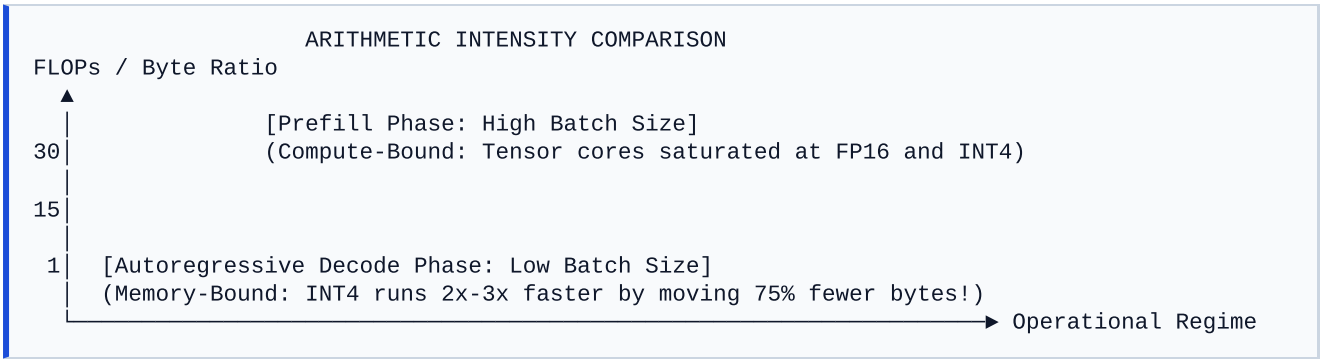
model_id = "meta-llama/Meta-Llama-3-8B-Instruct"

# 2. Instantiate pipeline with automated device mapping
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto"
)

# 3. Execute inference
inputs = tokenizer("Verify production deployment protocol:", return_tensors="pt").to("cuda")
with torch.no_grad():
    outputs = model.generate(**inputs, max_new_tokens=32)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

11.14 Mixed-Precision (FP16/BF16) vs. Quantized Inference

Understanding when to run full 16-bit precision versus low-bit quantized models requires analyzing compute intensity and operational requirements.



1. Arithmetic Intensity & Hardware Bottlenecks

Arithmetic Intensity measures the ratio of arithmetic operations to memory access:

$$\text{Arithmetic Intensity} = \frac{\text{Computational Operations (FLOPs)}}{\text{Memory Transferred (Bytes)}}$$

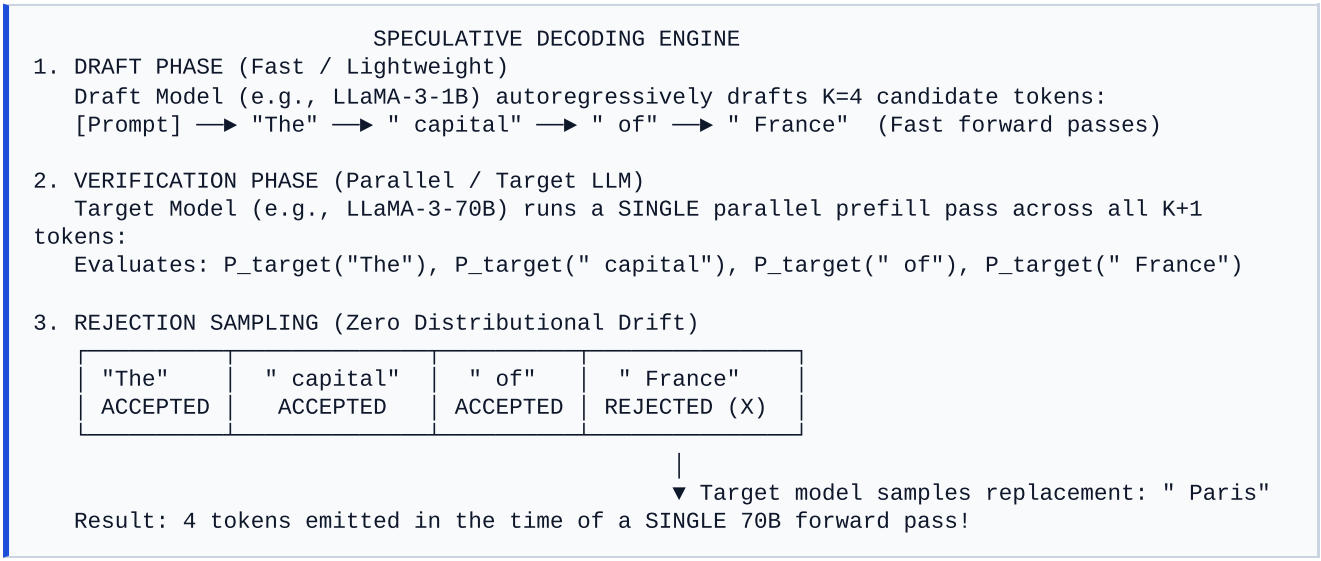
- **Low Batch Sizes (Single-Stream Serving / Batch = 1):** Arithmetic intensity is low (≈ 1 FLOP / Byte). Execution is limited by memory bandwidth. Moving 16-bit weights over the memory bus is $3\times$ to $4\times$ slower than moving 4-bit weights. **Quantization yields direct $2\times-3.5\times$ speedups.**
- **Large Batch Sizes (Batch ≥ 64 on Server GPUs):** Arithmetic intensity is high. Tensor cores are saturated with floating-point matrix multiplications. Weight loading latency is amortized across all tokens in the batch. Under this regime, the overhead of dequantizing 4-bit weights into float representations can degrade throughput, making **FP16 or native FP8** faster than INT4.

2. Decision Matrix

Operational Criterion	Full Precision (FP16 / BF16)	Quantized Inference (INT4 / AWQ / GGUF)
Numerical Fidelity	100% exact baseline representation.	98.5% – 99.5% fidelity (potential minor loss in complex reasoning).
Decoding Latency (B=1)	Slow (Limited by memory bus bandwidth).	**Fastest ($2\times-3\times$ speedup over FP16).**
Max Throughput ($B \geq 64$)	High (Optimal utilization of dense tensor cores).	Can encounter dequantization bottlenecks unless using native FP8/INT4 kernels.
Hardware Accessibility	Demands high-end multi-GPU clusters.	Runs on single commodity GPUs, Apple Silicon, or CPUs.
KV Cache Footprint	Standard FP16 memory consumption.	Easily pairs with quantized KV caches (FP8/INT4 KV cache).

11.15 Speculative Decoding: Complementary Acceleration

Quantization compresses model memory footprints; **Speculative Decoding** (Leviathan et al., 2022; Chen et al., 2023) directly targets the memory-bandwidth bottleneck of autoregressive decoding without altering the model's output distribution.



1. Algorithmic Mechanics

Autoregressive decoding is memory-bandwidth bound because reading 70 billion parameters from HBM yields only a single token:

> Cost(1 token from Target Model) ≈ Read Time of Full Target Model Weights

Speculative decoding pipelines two models:

1. **Draft Model (M_{draft}):** A small, high-throughput model (e.g., an 8-bit 1B parameter model) that quickly generates K candidate tokens sequentially.
2. **Target Model (M_{target}):** The primary production model (e.g., an FP16 70B parameter model).
3. **Parallel Verification:** Because the K candidate tokens are already generated, the target model evaluates them all **in a single forward pass** via causal mask prefilling.
4. **Rejection Sampling Gate:** For each drafted token $\tilde{x}_i$, the token is accepted with probability:

   $$\alpha = \min\left(1, \quad \frac{P_{\text{target}}(\tilde{x}_i \mid x_{<i})}{P_{\text{draft}}(\tilde{x}_i \mid x_{<i})}\right)$$

   - If token $\tilde{x}_i$ is accepted, generation advances to $\tilde{x}_{i+1}$.
   - If token $\tilde{x}_i$ is rejected, the loop halts. A replacement token is sampled from the adjusted residual distribution:

     $$P_{\text{adjusted}}(x) = \frac{\max(0,\ P_{\text{target}}(x) - P_{\text{draft}}(x))}{Z}$$

     and all subsequent draft tokens $\tilde{x}_{i+1 \ldots K}$ are discarded.

2. Key Mathematical Invariant

- **Exact Distributional Equivalence:** Speculative decoding is not an approximation. It is mathematically guaranteed that **the output probability distribution of accepted tokens matches the target model's output distribution exactly**.
- **Complementary to Quantization:** Speculative decoding can be combined with quantization. For example, a 4-bit quantized 70B target model verified against an INT4 1B draft model can achieve total generation speeds of ***80–120+* tokens/sec** on a single workstation GPU.

11.16 Hardware-Specific Architectural Profiling

Inference optimization must account for the physical interconnects, memory architectures, and compute units of the host hardware.

```
┌──────────────────────────────────┬──────────────────────────────────┬──────────────────────────────────┐
│       CONSUMER WORKSTATION       │      DATA-CENTER ENTERPRISE      │          APPLE SILICON           │
│        (1x RTX 4090 24GB)        │       (8x H100 80GB SXM5)        │         (M4 Max / Ultra)         │
├──────────────────────────────────┼──────────────────────────────────┼──────────────────────────────────┤
│ • Bus: PCIe 4.0 x16 (31.5 GB/s)  │ • Bus: NVLink 4 (900 GB/s)       │ • Unified Memory Architecture    │
│ • VRAM: 24 GB GDDR6X             │ • VRAM: 640 GB HBM3              │ • RAM: Up to 128GB–192GB         │
│ • Bandwidth: 1,008 GB/s          │ • Bandwidth: 26,800 GB/s         │ • Bandwidth: 400–800 GB/s        │
│ • Bottleneck: PCIe transfers &   │ • Bottleneck: Inter-node scale;  │ • Bottleneck: Compute cores;     │
│   strict 24GB capacity ceiling   │   high fixed hardware costs      │   slower raw matrix FLOPS        │
└──────────────────────────────────┴──────────────────────────────────┴──────────────────────────────────┘
```

1. Consumer GPUs (NVIDIA RTX 3090 / 4090 – 24GB VRAM)

- **The 24GB Memory Wall:** A 70B parameter model in FP16 requires 140 GB of VRAM, making it impossible to fit in a single consumer GPU without massive quantization or CPU offloading.
- **PCIe Bus Bottleneck:** Connecting a consumer GPU to system RAM over PCIe Gen 4 provides only $\sim$ *31.5* GB/s of transfer bandwidth. Offloading layers to system RAM creates a severe transfer bottleneck that slows decoding down to single-digit tokens per second.
- **Optimization Strategy:** Models must fit **entirely within the 24GB VRAM**.
  - Use **EXL2 or AWQ at 3.0 to 3.5 bits per weight** ($\sim$ *18* GB total model footprint).
  - This leaves 6 GB of VRAM available for a 4-bit quantized KV cache (`vllm --kv-cache-dtype fp8`), maintaining single-user decoding speeds of *30–40* tokens/sec.

2. Data-Center GPUs (NVIDIA A100 / H100 / Blackwell Clusters)

- **Massive Bandwidth & NVLink Fabrics:** NVIDIA H100 SXM5 cards provide *3,350* GB/s of HBM3 bandwidth per GPU, linked via fourth-generation NVLink running at *900* GB/s bidirectional interconnect speeds.
- **Native FP8 Hardware Acceleration:** Hopper architectures feature second-generation Transformer Engines supporting **native FP8 matrix-multiply tensor cores**.
- **Optimization Strategy:**
  - Avoid low-bit integer conversions (INT4) for large-scale multi-user deployments. Use **native FP8 (E4M3 format)**, which halves memory bandwidth consumption while running on high-throughput native tensor cores without runtime dequantization overhead.
  - Implement **Tensor Parallelism (*TP = 2, 4, 8*)** across NVLink to distribute the KV cache and parameter storage across the cluster.

3. Apple Silicon & Modern CPU Architectures

- **Unified Memory Architecture (UMA):** Apple M-series chips (M2/M3/M4 Max and Ultra) share high-bandwidth LPDDR5X memory directly between CPU cores, the Metal GPU, and the Neural Engine:
  - An Apple Silicon Mac configured with 128 GB or 192 GB of unified memory can hold an unquantized 70B model or a 4-bit 405B model entirely within addressable memory.
  - Memory bandwidth reaches ***400–819* GB/s**, outpacing conventional dual-channel x86 CPU memory buses (≈ *60–90* GB/s).
- **Optimization Strategy:**
  - Deploy models via **GGUF containers through `llama.cpp` using Metal kernels**.
  - Keep models within quantized tiers (**`Q4_K_M` or `Q5_K_M`**) to balance bandwidth utilization with inference fidelity.
  - For x86 CPU servers, compile `llama.cpp` with native **AVX-512 and AMX (Advanced Matrix Extensions)** support, matching worker threads strictly to physical CPU cores to avoid thread contention.

Technical Summary of Module 11 Part 2

Section	Core Mechanism	Mathematical Formulation / Invariant	Primary Failure Mode Addressed
11.9 AWQ Theory	Activation-aware equivalent scaling	$Y = (X S^{-1})(S W)$; $S = \text{diag}(s_X^{\gamma})$	Quantization error amplification caused by activation outliers
11.10 AWQ vs. GPTQ	First-order activation scaling vs. second-order OBS updates	AWQ preserves weight distributions; GPTQ mutates weights via H^{-1}	Hessian degradation on outliers and domain overfitting in GPTQ
11.11 GGUF & K-Quants	Self-describing `mmap` binary container; hierarchical super-blocks	`Q4_K_M` : 256-weight blocks with 6-bit scales	Metadata brittleness and scale storage overhead in legacy formats
11.12 Deployment Routing	Mapping quantization schemes to target hardware profiles	Server: FP8/AWQ; Consumer: EXL2/AWQ; Edge/Mac: GGUF	Sub-optimal hardware pairing (e.g., running GGUF on data-center clusters)
11.13 `bitsandbytes`	Vector-Wise Outlier Decomposition; NF4 dequantization	$Y = Y_{int8} + Y_{fp16}$ on channels $\lvert x\rvert \geq 6.0$	Degradation and perplexity collapse on models > 6.7B
11.14 Precision Regimes	Arithmetic intensity thresholding: Memory-bound vs Compute-bound	$\text{Intensity} = \text{FLOPs} / \text{Bytes}$; Low batch $\Longrightarrow$ Memory-bound	Inefficient serving configs across batch-size regimes
11.15 Speculative Decoding	Rejection sampling verification over small draft models	$\alpha = \min(1, P_{target}(\tilde{x}) / P_{draft}(\tilde{x}))$; Zero distribution shift	Memory-bound latency bottleneck in autoregressive single-token decode
11.16 Hardware Profiles	Hardware constraints: PCIe vs NVLink vs Apple Unified RAM	Fit 70B into 24GB via ~ 3.5 bpb; leverage H100 FP8 engines	Out-of-memory crashes and bus transfer saturation